

Math GRPO 训练 TensorBoard 完整分析报告

分析对象：primus47a424267ba73c850de046fe3z 的第 1—90 步训练记录。
数据时间范围：2026-09-15 00:38:47—15:07:21，北京时间。
本报告依据已读取的 TensorBoard 原始标量与运行提交的代码定义整理，不代表第 90 步之后的最新任务状态。

目录

1. 报告范围与实验背景	8. 长度与空回答指标
2. 结论摘要	9. 计算负载均衡 global_seqlen
3. 训练流程与统计口径	10. 耗时指标 timing
4. 奖励指标 reward	11. 吞吐与内存指标 perf
5. 优势与回报指标 critic	12. 训练进度 training
6. 策略更新指标 actor	13. 跨指标分析与后续观察建议
7. 回答级诊断 train_samples	14. 代码依据与解释边界
	15. 附录：96 个指标完整清单

1. 报告范围与实验背景

1.1 任务信息

项目	内容
任务 ID	primus47a424267ba73c850de046fe3z
任务名称	mopd_idea; verl_rl_math; Qwen3-4b-sft-mix-epoch4; fix-c354269
代码提交	c35426965eabc4f1af17357d4e130af1e2149537
启动脚本	hlg/mopd-idea/run_rl_math.sh
训练入口	python3 -m verl.trainer.main_ppo_sync
算法	GRPO, 纯数学强化学习
初始模型	Qwen3-4B, 任务选择的 SFT epoch4 checkpoint
训练与生成	FSDP actor + 同卡 vLLM rollout
硬件	单节点、8 张 L20X GPU
每批题目数	128
每题生成数	6
每步回答数	$128 \times 6 = 768$
Prompt 长度上限	1,024 token
Response 长度上限	31,000 token
学习率	1e-6, 本次记录中保持不变
蒸馏	关闭, 无教师模型
Reward KL / actor KL loss	均关闭
熵正则系数	0, 但仍记录 entropy
验证	本次配置未启用训练前验证和周期验证
计划训练步数	135
本报告分析范围	第 1—90 步

页面入口：

- [Primus 任务页面](#)
- [本任务 TensorBoard](#)

1.2 数据与分析方法

本次读取了：

- 13 个顶层指标分组；
- 96 个 scalar 标签；
- 每个标签连续第 1—90 步，共 8,640 个原始标量点；
- 未发现缺步或 NaN/Inf 等非有限值。

报告约定：

- **前窗口**：第 1—10 步原始值的算术平均。
- **后窗口**：第 81—90 步原始值的算术平均。
- **第 90 步**：该步的原始值，不是平滑值。
- **全程**：仅指本报告覆盖的第 1—90 步。
- **范围**：某指标在这 90 个步骤中的最小值至最大值。

TensorBoard 页面 smoothing 为 0.6，但本报告的数值比较不使用平滑后的曲线。前后窗口对比主要用于减少单步噪声，不能替代固定数据集评估或统计显著性检验。

对于每步都是 768 条回答的比例指标，十步比例的平均也等于这十步合并后的回答比例。但同一道题的六次生成并非独立题目，不能把 7,680 条回答当成 7,680 道独立试题。

2. 结论摘要

2.1 总体判断

这次训练在所观察的 90 步内数值稳定、持续产出有效批次；格式遵循明显改善，数学训练正确率总体有所上升，尚不能据此判断独立评测或泛化能力提升。

核心指标	前窗口	后窗口	第 90 步	解读
数学回答正确率	67.66%	70.48%	73.96%	前后窗口提高 2.83 个百分点，但存在波动
Think 格式合格率	96.12%	99.64%	99.74%	格式已接近饱和
综合奖励	0.7335	0.7631	0.7911	正确率与格式均有贡献
平均回答长度	9,398.6	9,600.7	8,798.6	小幅增加，没有明显长度失控
Actor token 平均熵	0.2928	0.2725	0.2693	输出分布更集中
梯度范数	0.03785	0.03451	0.03492	稳定，远低于默认裁剪阈值 1
回答截断比例	0.156%	0.273%	0.130%	绝对比例很低
每步耗时	554.8 秒	591.2 秒	524.0 秒	后窗口略慢，主要在生成阶段
吞吐	1,650.0	1,589.3	1,636.5	单位 token/s/GPU

2.2 最重要的解释边界

- `critic/*` 不意味着本次训练了 critic 模型。这次是 GRPO，该前缀沿用框架命名。
- `ppo_kl=0`、`pg_clipfrac=0` 不意味着模型没有学习。它们记录的是本次 optimizer step 之前的比较。
- `distill_kl_fwd_*=0` 是未启用教师时的占位值。不是测得师生完全一致。
- `student_mean_prob≈0.868` 不是答题正确率 86.8%。它衡量已生成 token 的概率。
- MFU 大于 1 不能解释为真实硬件利用率超过 100%。当前硬件峰值匹配与 FLOPs 估算口径需要校准。
- 训练奖励改善不等于泛化能力已改善。当前没有独立验证曲线支撑这一结论。

3. 训练流程与统计口径

3.1 高层流程



3.2 统一符号

符号	含义
N	一步参与统计的回答数量，本次为 768
P_i	第 i 条回答对应的 prompt 长度
L_i	第 i 条回答的 response 长度
m_{it}	第 i 条回答第 t 个 token 是否参与相应统计，通常为 0 或 1
$M_i = \sum_t m_{it}$	回答的有效统计 token 数
R_i	单条回答的最终任务奖励
A_i	单条回答的 GRPO 优势
F_i	Think 格式是否合格的指示变量
C_i	数学答案是否正确的指示变量
p_{itv}	在当前位置，模型为词表 token v 分配的概率
G	GPU 数量，本次为 8

3.3 回答平均与 token 平均

回答平均：每条回答一票。

$$\text{Mean}_{\text{sample}}(x) = \frac{1}{N} \sum_{i=1}^N x_i$$

Token 平均：每个有效 token 一票，长回答权重更大。

$$\text{Mean}_{\text{token}}(x) = \frac{\sum_{i,t} m_{it} x_{it}}{\sum_{i,t} m_{it}}$$

例如，一条 1,000 token 的回答与一条 10,000 token 的回答，在回答平均中权重相同；在 token 平均中，后者权重是前者的 10 倍。

本报告中的常见分工：

统计方式	典型指标
回答平均	reward score/acc、train_samples/advantage_mean、student_entropy_mean
Token 平均	actor/entropy、critic/advantages/mean、默认 token-mean PG loss
Micro-batch 内平均后再归约	PPO clipfrac、ppo_kl 等诊断项
每设备分区 token 总量	global_seqlen/*

有效样本统计通常排除为分布式对齐补出的 padding 行；性能与负载统计可包含实际参与计算的补齐行。本次 768 条回答与 8 个 actor DP 分区整除，当前数据未呈现由补齐引起的口径差异。

4. 奖励指标 reward：26 项

4.1 两个前缀的关系

- reward/all/*：所有任务领域合并后的奖励统计。
- reward/math/*：只统计数学领域。

本次只有数学数据，两组 13 项指标在第 1—90 步的原始值完全一致。因此以下 13 个后缀对应总共 26 条曲线。

4.2 单条回答的奖励公式

$$R_i = 0.2F_i + 0.8C_i$$

格式	答案	奖励
不合格	错误	0
合格	错误	0.2
不合格	正确	0.8
合格	正确	1

格式检查使用以回答开头为锚点的 `think` 标签规则，允许前导空白、跨行和大小写变化，要求 `<think>...`
`</think>` 闭合；不检查推理内容质量，`think` 内容也可以为空。

数学答案从全文最后一个 `\boxed{...}` 提取，支持括号嵌套，再经过答案归一化、符号等价等检查。它不是直接的全文字符串匹配，也不验证每一步推导。最后一个 `box` 损坏时，不应把它理解为必然会回退到之前正确的 `box`。

4.3 逐项定义与分析

下表指标名均需加上 `reward/all/` 或 `reward/math/` 前缀。

指标后缀	定义或公式	本次数据及分析
<code>samples</code>	当前批次参与奖励统计的回答数 N	恒为 768，不是 128 道题
<code>score</code>	$\frac{1}{N} \sum_i R_i$	前后窗口 0.7335 → 0.7631，第90步 0.7911
<code>acc</code>	$\frac{1}{N} \sum_i C_i$	前后窗口 67.66% → 70.48%；第90步 568/768=73.96%
<code>format_score</code>	$\frac{1}{N} \sum_i F_i$	第1步 83.72%，第90步 766/768=99.74%，改善明显
<code>answer_extracted</code>	成功提取 <code>boxed</code> 答案的回答数 / N	前后窗口 99.47% → 99.61%；第90步 766/768，但提取不等于正确
<code>checker_timeout</code>	外层判题进程超时数 / N	全程 0；默认外层超时为30秒，不覆盖所有内部检查超时概念
<code>constraint_ratio</code>	通用奖励结构的约束满足率字段	数学任务未使用，0 是占位值
<code>test_pass_ratio</code>	通用奖励结构的代码测试通过率字段	数学任务未使用，0 是占位值
<code>termination_known</code>	已知结束原因的回答数 K/N	恒为 1，结束原因完整
<code>known_termination_count</code>	已知结束原因的回答数 K	恒为 768
<code>is_truncated</code>	已知结束样本中， <code>loop</code> 截断或 <code>backend= length</code> 的比例	全程平均 0.234%；第90步 1/768=0.130%
<code>stopped_normally</code>	已知结束样本中， <code>backend= stop</code> 且未 <code>loop</code> 截断的比例	后窗口 99.73%；第90步 767/768=99.87%

指标后缀	定义或公式	本次数据及分析
generation_aborted	已知结束样本中， backend= abort/aborted 的比例	全程 0

终止原因已知的条件是：存在 loop 截断，或 backend 原因为 stop/length/abort/aborted。未知时，后三类终止比例的单样本值为 NaN，聚合会排除这些未知值，而不是强行记为正常或异常。本次全部已知，分母均为 768。

奖励聚合按各字段的有限数值分别求平均，并排除 padding 样本。外层数学判题超时，正确性和答案提取标志为 0，但仍可能保留格式奖励。

4.4 奖励改善来自哪里

$$\Delta \overline{R} = 0.2 \Delta \overline{F} + 0.8 \Delta \overline{C}$$

前后窗口中：

$$0.02964 \approx 0.2 \times 0.03516 + 0.8 \times 0.02826$$

因此：

- 格式改善贡献约 0.00703 的奖励提升；
- 正确率改善贡献约 0.02260；
- 奖励提升并不只是“学会了格式”。

不过 acc 是回答级正确率，不是 pass@6。同题六条中至少一条答对的题目比例，需要按题目分组另行计算，不能从当前 acc 直接读取。

代码依据： plugin/open_mopd_rewards/common.py:56、 plugin/open_mopd_rewards/common.py:72、 plugin/open_mopd_rewards/common.py:98、 plugin/open_mopd_rewards/math_reward.py:14。

5. 优势与回报指标 critic：12 项

5.1 前缀不代表启用了 critic 模型

本次使用 GRPO，不需要通过价值模型估计基线。 critic/* 是框架沿用的统计名称，不能从这个前缀推断本次额外训练了 critic。

5.2 GRPO 优势公式

对同一道题的六个回答：

$$\mu_g = \frac{1}{6} \sum_{j=1}^6 R_j$$

$$s_g = \sqrt{\frac{\sum_{j=1}^6 (R_j - \mu_g)^2}{5}}$$

$$A_i = \frac{R_i - \mu_g}{s_g + 10^{-6}}$$

这里 `torch.std` 默认采用 `correction=1`，方差分母为 $6 - 1 = 5$ ，epsilon 加在标准差外面。更严格地说，这是样本方差的无偏校正，不应把标准差本身称为严格无偏估计。

直观上，优势衡量的是“这条回答相对于同一道题的其他回答好多少”，不是单独的绝对分数。

在当前单轮实现中，每条回答的有效 token 共享该回答的优势；本 GRPO 路径直接返回相同的 `advantages` 与 `returns`。

5.3 四个指标族

下表每一行都包含 `mean`、`max`、`min` 三条指标，总计12项。

指标族	计算口径	第90步 mean / max / min
<code>critic/score/{mean,max,min}</code>	每条回答先计算 $S_i = \sum_t \text{token_level_scores}_{it}$ ，再对非空回答统计	0.7911 / 1 / 0
<code>critic/rewards/{mean,max,min}</code>	对 $R_i = \sum_t \text{token_level_rewards}_{it}$ 做相同统计	0.7911 / 1 / 0
<code>critic/advantages/{mean,max,min}</code>	在所有有效 response token 中统计优势	-0.04555 / +2.041235 / -2.041236
<code>critic/returns/{mean,max,min}</code>	在所有有效 response token 中统计 returns	与 advantages 完全相同

细节：

- `score/reward` 的单回答求和本身不乘 `response mask`；随后排除长度为0的回答。
- `advantages/returns` 的均值、极值只在有效 `response mask` 上统计。
- 本次 reward KL 关闭，因此 `score` 与 `rewards` 的对应曲线相同。
- 本次 GRPO 直接令 `returns=advantages`；不要把这里的 `returns` 当作另一套独立计算的原始折扣回报。

5.4 为什么 token 平均优势为负

当回答内优势相同时：

$$\bar{A}_{token} = \frac{\sum_i M_i A_i}{\sum_i M_i}$$

组内中心化保证的是：

$$\sum_{i \in g} A_i \approx 0$$

而不是：

$$\sum_{i \in g} M_i A_i = 0$$

本次 `critic/advantages/mean` 的前后窗口为 $-0.05607 \rightarrow -0.04780$ ，第90步为 -0.04555 。与此同时，回答平均优势接近0。

二者不矛盾。它说明在当前批次中，相对较差回答的有效长度权重更大；不能据此认定优势归一化失败。

5.5 为什么极值接近 ± 2.041

六个回答中，若一个高分、五个同样低分，忽略很小的 epsilon：

$$A_{\max} = \frac{5}{\sqrt{6}} \approx 2.04124$$

例如一个1分、五个0.2分：

$$A_{\max} = \frac{5 \times 0.8/6}{0.8/\sqrt{6} + 10^{-6}} \approx 2.041235$$

反过来，一个低分、五个同样高分，产生负极值。日志中的极小数值差异可来自浮点计算。

这是组内标准化的结果，不是设置了 ± 2.041 的裁剪阈值。

代码依据：`verl/trainer/ppo/core_algos.py:304`、`verl/trainer/ppo/metric_utils.py:123`。

6. 策略更新指标 actor：8 项，不含内存

6.1 当前 PPO 损失公式

定义当前训练前向相对于 old policy 的 logprob 差：

$$\delta_{it} = \text{clip}(\log \pi_{\theta}(a_{it}) - \log \pi_{old}(a_{it}), -20, 20)$$

$$r_{it} = e^{\delta_{it}}$$

默认普通 PPO clip 范围为 $[0.8, 1.2]$ 。省略下标，定义：

$$u = -Ar, \quad v = -A \operatorname{clip}(r, 0.8, 1.2), \quad w = \max(u, v)$$

代码还包含系数为3的 dual-clip：

$$\ell = \begin{cases} \min(w, -3A), & A < 0 \\ w, & A \geq 0 \end{cases}$$

本次没有额外 rollout IS 权重，默认聚合为 token-mean：

$$L_{PG} = \frac{\sum_{i,t} m_{it} \ell_{it}}{\sum_{i,t} m_{it}}$$

高层含义：提高好回答中 token 的概率，降低差回答中 token 的概率，同时限制某些过大的更新贡献。

6.2 逐项解释

指标	定义或公式	本次情况
actor/pg_loss	上述策略梯度损失 L_{PG}	前后窗口 0.05607 → 0.04780；第90步 0.04555
actor/loss	实际用于反向传播的总损失	本次熵正则、KL loss、蒸馏都关闭，因此与 pg_loss 基本相等
actor/pg_clipfrac	有效 token 中 $v > u$ 的比例	全程 0，普通 PPO clip 没有改变目标
actor/pg_clipfrac_lower	有效 token 中 $A < 0$ 且 $w > -3A$ 的比例	全程 0，dual-clip 未触发，不是简单的 $r < 0.8$ 比例
actor/ppo_kl	micro-batch 内 $\operatorname{Mean}_{token}(-\delta)$ ，再跨 micro-batch/DP 归约	全程 0；比较 old/current policy，不是与参考模型的 KL
actor/grad_norm	$\ g\ _2 = \sqrt{\sum_k (\partial L / \partial \theta_k)^2}$	全程 0.0287–0.0425；第90步 0.03492
actor/lr	优化器首参数组的学习率，当前为常数 $\eta_t = 10^{-6}$	全程 1e-6；图形自动缩放不能作为学习率为0的证据
actor/entropy	全词表熵的有效 token 平均	前后窗口 0.2928 → 0.2725 nat，第90步 0.2693

熵的公式是：

$$H_{it} = - \sum_{v \in \mathcal{V}} p_{itv} \ln p_{itv}$$
$$H_{actor} = \frac{\sum_{i,t} m_{it} H_{it}}{\sum_{i,t} m_{it}}$$

这里是完整词表分布熵，不是实际采样 token 的负 logprob。自然对数对应单位 nat。

补充统计口径：

- `loss/pg_loss` 经过 micro-batch 缩放和 DP 归约，对应本次全局有效 response token 平均。
- `clipfrac`、`clipfrac_lower`、`ppo_kl` 先在 micro-batch 内求 token 平均，再对 micro-batch / DP 结果平均，不严格等于全局 token 加权均值。本次三者均为0，这个差异不影响观察结论。
- `ppo_kl` 是有符号的采样近似，通常不能要求每一步都非负；它不是完整分布上的精确 KL。
- `grad_norm` 是全 mini-batch 梯度累积并同步后、裁剪前的范数，不是参数更新量。
- `lr` 在 scheduler 推进后读取首参数组；当前 constant scheduler、无 warmup，所以保持1e-6。

6.3 为什么 KL 与 clipping 全为0，模型仍然会更新

当前每步只有一个覆盖整批数据的 mini-batch：

$$128 \times 6 = 768$$

同时 PPO epoch 为1，old logprob 在 actor 更新前重算。训练顺序是：

```
更新前 actor 重算 old logprob
      |
      v
各 micro-batch 前向并反向累积梯度
      |
      v
全部完成后执行一次 optimizer.step()
```

因此，损失诊断计算时参数尚未发生这一轮 optimizer 更新，当前前向与 old policy 一致时：

$$r = 1, \quad \text{ppo_kl} = 0, \quad \text{clipfrac} = 0$$

此时损失数值化简为：

$$L_{PG} = -\text{Mean}_{\text{token}}(A)$$

但梯度并不因此为0：

$$\nabla_{\theta} L = -\frac{\sum_{i,t} m_{it} A_i \nabla_{\theta} \log \pi_{\theta}(a_{it})}{\sum_{i,t} m_{it}}$$

数据交叉验证：90步内，`actor/pg_loss` 与 `-critic/advantages/mean` 的最大绝对差异约为 **2.6e-8**，与上述解释一致。

因此：

- 全0的 KL/clipping 不是“训练停了”的证据。
- 它们不是 optimizer 更新之后重新测量的模型变化。
- PG loss 不是监督学习交叉熵，不能仅凭数值接近0就判定训练更好。
- 默认梯度裁剪阈值为1，当前最大梯度范数约0.0425，没有接近这一阈值。

6.4 熵下降的含义

熵下降说明当前训练样本上下文中的输出分布更集中，与 `student_mean_prob` 上升相互印证。

它可能对应更稳定的格式和更确定的回答，也可能伴随探索减少。仅凭熵缓慢下降，不能判断模式坍塌；应与正确率、长度、组内答案多样性一起观察。

代码依据：`verl/trainer/ppo/core_algos.py:1329`、`verl/trainer/main_ppo_sync.py:1377`、`verl/trainer/main_ppo_sync.py:1565`、`verl/workers/engine/fsdp/transformer_impl.py:659`。

7. 回答级诊断 `train_samples`：10 项

7.1 统计范围与标准差

这些标量每步输出，从第1步开始，覆盖完整有效 batch。

- `VERL_LOG_SAMPLE_FREQ=5` 控制文本诊断打印频率。
- `VERL_LOG_MAX_GROUPS=16` 限制打印的题目组数。
- 两者都不把本次 TensorBoard 标量截成只统计少数打印样本。

本组使用 NumPy 总体标准差：

$$\text{Std}(x) = \sqrt{\frac{1}{N} \sum_i (x_i - \bar{x})^2}$$

这里分母为 N ，与 GRPO 六回答组内标准差的分母5不同。

7.2 逐项解释

表中名称省略 `train_samples/` 前缀。

指标	定义或公式	本次情况
<code>reward_mean</code>	$\frac{1}{N} \sum_i R_i$	与 <code>reward/math/score</code> 原始值相同；第90步 0.7911
<code>reward_std</code>	$\text{Std}(R_i)$	前后窗口 0.3791 $\rightarrow$ 0.3648 ；第90步 0.3521
<code>reward_max</code>	$\max_i R_i$	每步均为 1
<code>reward_min</code>	$\min_i R_i$	通常为 0 ，部分批次为 0.2 ；第90步为 0
<code>advantage_mean</code>	每条回答取第一个有效 token 的优势，再按回答平均	绝对值约在 2.3e-8 内，可视为 0
<code>advantage_std</code>	回答级优势的总体标准差	前后窗口 0.6276 $\rightarrow$ 0.5838 ；第90步 0.5532
<code>distill_kl_fwd_mean</code>	各回答 token 平均蒸馏诊断值的均值	本次没有教师， 全0是占位值
<code>distill_kl_fwd_max</code>	各回答 token 平均蒸馏诊断值的最大值	本次同样是 占位0 ，不是单 token 最大值
<code>student_mean_prob</code>	对实际生成 token 的旧 actor 概率，先回答内平均，再回答间平均	前后窗口 0.8611 $\rightarrow$ 0.8678 ；第90步 0.8670
<code>student_entropy_mean</code>	先平均每条回答的全词表熵，再回答间平均	前后窗口 0.2747 $\rightarrow$ 0.2569 nat ；第90步 0.2574

无有效 token 的回答在优势标量提取中记为0；概率和熵诊断则跳过没有有效 token 的回答。本次没有空回答，这不影响当前结果。

7.3 概率与熵的公式

$$\text{student_mean_prob} = \frac{1}{N_+} \sum_{i:M_i>0} \left[\frac{1}{M_i} \sum_t m_{it} \exp(\log \pi_{old}(a_{it})) \right]$$

$$\text{student_entropy_mean} = \frac{1}{N_+} \sum_{i:M_i>0} \left[\frac{1}{M_i} \sum_t m_{it} H_{it} \right]$$

其中 N_+ 为具有有效统计 token 的回答数。

解释要点：

- `student_mean_prob` 是已采样 token 的平均概率，不是答案正确率。
- 它是“先 exp，再平均”，不是“先平均 logprob，再 exp”。
- `student_entropy_mean` 与 `actor/entropy` 使用同一份更新前 entropy；前者每条回答等权，后者每个 token 等权。
- 错误回答也可能获得很高的 token 概率；确定性不等于正确性。

7.4 为什么蒸馏 KL 是占位值

有教师数据时，当前诊断代码近似计算：

$$k_i = \frac{1}{M_i} \sum_{t:m_{it}=1} [\log p_s(a_{it}) - \log \tilde{p}_T(a_{it})]$$

然后分别记录：

$$\text{distill_kl_fwd_mean} = \text{Mean}_i(k_i)$$

$$\text{distill_kl_fwd_max} = \max_i k_i$$

若教师 top-k 中找不到采样 token，代码以该位置 teacher top-k 的最小 logprob 替代，因此它不是精确的全词表 KL。也不能仅从 fwd 名称推断一个完整分布 KL 的计算方向与精确性。

本次没有教师数据，教师相关诊断没有被计算，但返回结构将缺失的 KL 均值填为0，随后进入聚合。因此两个全0曲线没有师生差异方面的解释意义。

7.5 优势标准差下降意味着什么

一题六个回答全部同分时，该组优势全为0。只有组内奖励不同，才产生区分回答的优势信号。

因此 advantage_std 下降可能意味着同分组更多，而不一定意味着梯度计算出现问题。理论上，忽略 epsilon 且各组大小均为6时：

$$\text{Var}_{batch}(A) \approx \frac{5}{6} \times \text{组内奖励非恒定的题目组比例}$$

这个关系来自每个非恒定组的标准化优势平方和约为5。

但同分组既可能六条全对，也可能六条全错；仅靠此指标无法区分。当前格式接近饱和，也可能减少仅由格式差异贡献的优势信号。

代码依据：verl/trainer/main_ppo_sync.py:1664、verl/trainer/main_ppo_sync.py:2069、verl/trainer/main_ppo_sync.py:2186。

8. 长度与空回答指标：13 项

8.1 Prompt 长度：4 项

指标	公式	当前情况
prompt_length/mean	$\frac{1}{N} \sum_i P_i$	前后窗口 132.2 → 131.0；第90步 133.5
prompt_length/max	$\max_i P_i$	第90步 358；90步中最长 prompt 为 1,002
prompt_length/min	$\min_i P_i$	第90步 54；各步最短值范围 52–66
prompt_length/clip_ratio	$\#(P_i = 1024)/N$	全程 0

这里 clip_ratio 是“恰好达到配置上限”的比例，代码判断是等于而不是大于等于。

它不是数据预处理阶段被过滤或截断的比例。本次 filter_overlong_prompts=True，因此不能由 clip_ratio=0 推断原始数据里没有超长题目。

8.2 Response 长度：4 项

指标	公式	当前情况
response_length/mean	$\frac{1}{N} \sum_i L_i$	前后窗口 9,398.6 → 9,600.7；第90步 8,798.6
response_length/max	$\max_i L_i$	第90步 31,000；各步最大值范围 26,218—31,000
response_length/min	$\min_i L_i$	第90步 614；全程最短回答 415
response_length/clip_ratio	$\#(L_i = 31000)/N$	全程平均 0.234%，单步最高 0.911%=7/768

单步平均回答长度的范围为 8,653.5—10,851.9 token。

90步共69,120条回答记录，按截断比例还原为162条截断回答。本次 response_length/clip_ratio 与 reward/math/is_truncated 的全部原始点相同，但二者定义不同：一个看长度是否等于上限，一个看结束原因 / loop 标志，不能在其他实验里默认永远相等。

response_length/max 经常达到31,000，并不代表多数回答被截断。它只需要每批768条里有一条碰到上限就成立，应结合 clip_ratio 看比例。

8.3 非空回答长度与 aborted：5 项

令非空回答集合为：

$$I = \{i : L_i > 0\}$$

指标	定义或公式	当前情况
response_length_non_aborted/mean	$\sum_{i \in I} L_i / I $	与 response_length/mean 完全相同
response_length_non_aborted/max	$\max_{i \in I} L_i$	与普通 max 完全相同
response_length_non_aborted/min	$\min_{i \in I} L_i$	与普通 min 完全相同
response_length_non_aborted/clip_ratio	$\#_{i \in I}(L_i = 31000) / I $	与普通 clip_ratio 完全相同
response/aborted_ratio	$\#(L_i = 0) / N$	全程 0

这里的 non_aborted 实际由“长度不为0”判断，不直接读取后端 finish reason。

因此：

- `response/aborted_ratio`：空回答比例。
- `reward/math/generation_aborted`：后端报告 abort 的比例。

一个已经生成部分内容后中止的回答，可以被后者计入，而不被前者计入。本次二者都为0。

本入口的 `prompt/response` 长度来自 `TransferQueue` 序列真实长度，`nested tensor` 使用 `offsets` 差值，等长 `dense tensor` 使用实际宽度，不把动态补齐后的最大宽度误当作所有样本长度。

代码依据：`verl/trainer/main_ppo_sync.py:2391`、`verl/trainer/ppo/metric_utils.py:144`、`verl/trainer/ppo/metric_utils.py:249`。

9. 计算负载均衡 `global_seqlen`：6 项

9.1 指标对应的层级

这些指标不是单条序列长度，而是整批数据分配到各 actor DP 分区后的累计 token 数。

设每条序列长度为：

$$\ell_i = P_i + L_i$$

设：

- U_j ：均衡前，按原始顺序连续分块后，第 j 个分区的 token 总量；
- V_j ：均衡后，第 j 个分区的 token 总量；
- 本次 actor DP 分区数为8，每区通常96条回答。

9.2 逐项定义

指标	公式	第90步
<code>global_seqlen/min</code>	$\min_j U_j$	214,453
<code>global_seqlen/max</code>	$\max_j U_j$	2,002,556
<code>global_seqlen/minmax_diff</code>	$\max_j U_j - \min_j U_j$	1,788,103
<code>global_seqlen/balanced_min</code>	$\min_j V_j$	856,578
<code>global_seqlen/balanced_max</code>	$\max_j V_j$	858,054
<code>global_seqlen/mean</code>	$\sum_i \ell_i / 8$	857,487.875

均衡后的相对极差：

$$\frac{858054 - 856578}{857487.875} \approx 0.172\%$$

均衡前最重分区约是最轻分区的9.3倍；均衡后 token 总量非常接近，说明负载均衡发挥了作用。

9.3 与真实计算量、micro-batch 的关系

实际划分使用工作量近似：

$$W(\ell) = 24576\ell + \ell^2$$

并采用等样本数分区；日志仍然输出原始 token 数之和，而不是 W 的值。

因此：

- token 总量接近，不等于每卡所有实际计算、通信耗时绝对相同。
- `global_seqlen/max` 很大，不表示一次前向喂入了两百万 token。
- 这些是整批分区累计量，与单次 micro-batch 的 `ppo_max_token_len_per_gpu` 属于不同层级。

代码依据：`verl/trainer/main_ppo_sync.py:1351`、`verl/utils/seqlen_balancing.py:27`、`verl/utils/seqlen_balancing.py:257`。

10. 耗时指标 timing：9 项

10.1 阶段墙钟耗时 timing_s：6 项

所有值单位为秒，对应计时代码块的经过时间。

指标	测量内容	前窗口	后窗口	第90步
<code>timing_s/gen</code>	等待本批 rollout / 打分就绪，不是纯 GPU 解码时间	376.56	409.99	359.49
<code>timing_s/old_log_prob</code>	actor 重算旧策略 logprob、entropy 及相关处理	33.87	34.22	31.21
<code>timing_s/adv</code>	奖励整理、优势计算及数据写回	1.084	1.082	1.138
<code>timing_s/update_actor</code>	actor 前后向、梯度处理、优化器更新等	134.46	136.86	124.04
<code>timing_s/update_weights</code>	将训练后的权重同步到 rollout 引擎	5.49	6.18	4.93
<code>timing_s/step</code>	训练步骤整体计时	554.81	591.24	523.98

整体 step 计时包含权重同步和当步启用的 checkpoint 等，但不包括随后全部验证、指标计算、日志清理等操作。因此它既不是训练进程从启动到结束的总耗时，也不严格等于相邻 TensorBoard 点的墙钟间隔。

阶段时间相加也不必精确等于 step，还存在数据准备、控制逻辑等未单列时间。

10.2 最近窗口的耗时结构

采用总时长之比：

阶段占比 = \frac{\sum_{s=81}^{90} t_{阶段,s}}{\sum_{s=81}^{90} t_{step,s}}

阶段	占比
生成等待	69.34%
Actor 更新	23.15%
Old logprob	5.79%
权重同步	1.05%
优势计算	0.18%
其他未单列开销	约0.50%

当前主要耗时在 rollout 等待阶段。若后续优化速度，应优先调查生成长度分布、长尾请求、并发与生成引擎利用情况，而不是首先优化约1秒的优势计算。

10.3 单 token 摊消耗时：3 项

定义：

T_R = \sum_i L_i, \quad T = \sum_i (P_i + L_i)

指标	公式	前窗口	后窗口	第90步
timing_per_token_ms/gen	1000t_{gen}/T_R	0.05222	0.05542	0.05320
timing_per_token_ms/update_actor	1000t_{update}/T	0.01837	0.01830	0.01808
timing_per_token_ms/adv	1000t_{adv}/T	0.0001481	0.0001451	0.0001659

单位均为 ms/token。

注意：

- gen 的分母只有 response token。
- actor update 和 adv 的分母为 prompt+response token。
- 三者没有再除以 GPU 数。
- 这是整批并行工作摊到 token 上的时间，不是用户体验中的单请求逐 token 延迟。
- 当前实现没有输出 old_log_prob 对应的每 token 指标，缺失不表示其耗时为0。

前后窗口中，actor 每 token 耗时基本稳定，生成每 token 摊消耗时略升。单凭这些值不能确定是长尾序列变化、调度、引擎负载还是其他运行时因素造成。

代码依据：`verl/trainer/ppo/metric_utils.py:310`、`verl/trainer/main_ppo_sync.py:2545`、`verl/trainer/main_ppo_sync.py:2651`。

11. 吞吐与内存指标 perf：7 项，含 actor 内存

11.1 全局吞吐与 MFU：4 项

指标	公式或定义	当前情况
<code>perf/total_num_tokens</code>	$T = \sum_i (P_i + L_i)$	每步 674.6万–844.1万；第90步 6,859,903
<code>perf/time_per_step</code>	与 <code>timing_s/step</code> 相同	前后窗口 554.8 → 591.2秒，第90步 523.98秒
<code>perf/throughput</code>	$T / (G t_{step})$ ，本次 $G = 8$	前后窗口 1650.0 → 1589.3 token/s/GPU；第90步 1636.5
<code>perf/mfu/actor</code>	模型估算 FLOPs 吞吐 / GPU 理论 FLOPs 峰值 / GPU 数	全程 2.174–2.260，绝对值不能直接当作真实效率

吞吐口径：

- 同一道题生成六次，其 prompt 长度也会在序列计数中出现六次。
- `total_num_tokens` 是 batch 序列 token 数，不是把所有前向、反向重复访问的 token 次数累计起来。
- `throughput` 是整步平均、按 GPU 数归一化的吞吐，不是单独的 vLLM 生成吞吐。
- 性能统计分母采用实际完整计算 batch；若有分布式补齐样本，也可能计入。

11.2 MFU 的公式、硬件匹配与局限

MFU 使用：

$$\text{MFU} = \frac{F_{estimated}}{t_{engine} \times G \times F_{peak\ per\ GPU}}$$

其中：

- $F_{estimated}$ ：按模型结构与序列长度估算的训练浮点运算量；
- t_{engine} ：engine mini-batch 训练计时，主要包括前后向、梯度处理和 optimizer 更新，不含 rollout 或 old logprob 重算；
- $F_{peak\ per\ GPU}$ ：代码设备表给出的每卡理论 FLOP/s；
- G ：分布式 world size，本次8。

对于当前 Qwen3 文本模型，代码使用 Qwen2 路径的估算器，其形式为：

$$F_{estimated} = 6D \sum_i \ell_i + 6d n_q n_l \sum_i \ell_i^2$$

$$D = n_l [3HI + 2Hd(n_q + n_{kv})] + 2VH$$

这里 H 为隐藏维度， I 为 FFN 中间维度， V 为词表大小， n_l 为层数， d 为 head 维度， n_q, n_{kv} 分别为 attention heads 与 KV heads； D 是估算器中的线性计算规模项，不当当作经过逐算子校准的真实操作量。

本次最重要的硬件口径问题：

- 1. 设备识别按名称子串匹配。
- 2. 代码表中有 `L20: 119.5e12`，没有独立的 L20X 条目。
- 3. L20X 会匹配 L20，使用119.5 TFLOP/s作为每卡峰值。
- 4. 最终 MFU 没有乘100，数值2.2是无量纲2.2倍，按比例相当于220%，不是2.2%。

这不意味着硬件真实利用率超过100%。它说明模型 FLOPs 估算、硬件峰值与计时口径的组合不适合直接作为本设备上的绝对效率解释。

代码能确认 L20X 被匹配到 L20 的口径；但本报告没有通过远端逐算子 profiling 和硬件标定证明偏差的唯一来源。

建议：当前 MFU 只保留为估算诊断值，不用它宣传实际 GPU 利用率或直接跨硬件比较效率。

11.3 Actor GPU/CPU 内存：3 项

尽管指标名写 `gb`，实际除数是 1024^3 ，因此单位是 GiB：

$$\text{GiB} = \frac{\text{bytes}}{1024^3}$$

指标	精确定义	当前情况
<code>actor/perf/max_memory_allocated_gb</code>	PyTorch 进程设备分配器记录的张量分配历史峰值	第1步 50.88 GiB，第90步 55.46 GiB，后窗口保持不变
<code>actor/perf/max_memory_reserved_gb</code>	PyTorch 缓存分配器保留显存的历史峰值	54.76 → 66.48 GiB，后窗口保持不变
<code>actor/perf/cpu_memory_used_gb</code>	<code>psutil.virtual_memory().used</code> ，系统可见已用内存	全程 70.02—76.25 GiB，第90步 72.66 GiB

Allocated 与 reserved 的区别

- allocated：分配给张量的显存。
- reserved：缓存分配器向设备保留的显存，可能包含暂时空闲、可复用的缓存。

因此 reserved 大于 allocated 正常，差额不全是活跃模型张量。

为什么峰值曲线阶梯上涨不等于泄漏

本训练路径没有每步重置 CUDA 峰值计数，所以这些曲线是自计数器上次重置以来的历史高水位，不是每一步的独立峰值。

更长的序列、不同 micro-batch 打包或初始化阶段，都可能抬高历史高水位。当前后窗口的平台化说明已记录峰值没有继续刷新，但不能单凭峰值曲线断言所有内存行为都正常。

聚合范围不能误读

这些性能量在 DP metric gather 之后添加；当前仅包含 metadata 的返回结果合并时保留首个 rank，本次对应 rank0。

因此：

- GPU 峰值不是8卡最大值，也不是8卡总和。
- CPU 数字不是 actor 单进程 RSS，也不是每个 actor 的 RSS 求和。
- Actor 进程内的 PyTorch 数字不包括同卡全部 vLLM 等独立进程的总显存。
- 不能由 reserved≈66 GiB 推断整卡总共只用了66 GiB。

代码依据： `verl/trainer/ppo/metric_utils.py:353`、 `verl/workers/engine_workers.py:183`、
`verl/workers/engine_workers.py:210`、 `verl/utils/flops_counter.py:22`、
`verl/utils/flops_counter.py:89`、 `verl/utils/tensordict_utils.py:221`。

12. 训练进度 training：5 项

指标	定义	当前情况
<code>training/global_step</code>	一次完整 rollout→训练 batch 的迭代编号，从1开始	本次读取 1—90，无缺步；计划135步
<code>training/epoch</code>	数据加载器外层 epoch 索引，从0开始	全程 0，表示第一轮，不是没有训练
<code>training/num_turns/mean</code>	有效回答携带的对话轮次标记均值	恒为 2
<code>training/num_turns/max</code>	对话轮次标记最大值	恒为 2
<code>training/num_turns/min</code>	对话轮次标记最小值	恒为 2

一般情况下 `global_step` 不等于内部 optimizer 子步总数，因为一轮可以有多个 mini-batch 或 PPO epochs；本次配置每轮只有一次更新。

当前 single-turn agent 直接将 `num_turns=2`，约定对应用户与助手的单轮交互。因此这三个常数不是两次思考、两次 rollout 或两次工具调用的证据。

代码依据： `verl/trainer/main_ppo_sync.py:2419`、 `verl/trainer/main_ppo_sync.py:2434`、
`verl/experimental/agent_loop/single_turn_agent_loop.py:79`。

13. 跨指标分析与后续观察建议

13.1 十步窗口走势

以下每行都是对应十步的原始指标平均，不是 TensorBoard 平滑值。

步数	正确率	格式合格率	综合奖励	平均回答长度	Actor 熵	每步秒数
1—10	67.66%	96.12%	0.7335	9,398.6	0.2928	554.8
11—20	68.19%	99.24%	0.7440	9,512.5	0.2906	573.1
21—30	66.85%	99.48%	0.7338	9,766.5	0.2877	574.8
31—40	68.36%	99.40%	0.7457	9,438.7	0.2814	547.9
41—50	68.74%	99.44%	0.7488	9,600.8	0.2787	570.4
51—60	69.65%	99.47%	0.7561	9,602.5	0.2776	568.3
61—70	69.78%	99.57%	0.7574	9,660.6	0.2699	572.7
71—80	72.55%	99.71%	0.7798	9,507.7	0.2728	555.9
81—90	70.48%	99.64%	0.7631	9,600.7	0.2725	591.2

可见：

- 格式改善主要发生在早期，之后在99%以上小幅波动。
- 数学正确率总体上行，但不是单调；不能只取第90步73.96%作为总体水平。
- 熵总体下降，后段也存在小幅反弹，而不是每步单调下降。
- 回答长度一直在同一量级波动，没有出现明显持续膨胀。
- 最近窗口生成等待更慢，但 actor 每 token 更新时间基本稳定。

13.2 已验证的重复关系与统计关系

关系	90步数据检查结果	原因
<code>reward/all/* = reward/math/*</code>	对应13项完全相同	只有数学数据
<code>critic/rewards/* = critic/score/*</code>	对应项相同	未启用 reward KL
<code>critic/returns/* = critic/advantages/*</code>	对应项相同	当前 GRPO 返回相同 tensors
<code>train_samples/reward_mean = reward/math/score</code>	原始值完全相同	都是完整批次回答奖励平均
<code>response_length_non_aborted/* = response_length/*</code>	对应项相同	没有零长度回答
<code>response_length/clip_ratio = reward/math/is_truncated</code>	当前原始值相同	本次长度达到上限与截断结束原因一致，定义仍不同
<code>perf/time_per_step = timing_s/step</code>	相同	同一计时值的不同指标名
<code>actor/pg_loss ≈ - critic/advantages/mean</code>	最大误差约2.6e-8	当前更新前 ratio≈1、token-mean、无额外项

score/reward 在不同代码路径间也可能有约1e-7量级的浮点舍入差异，不应把它理解为奖励逻辑不一致。

13.3 当前可以支持的结论

- 1. 训练记录连续且数值有限。96项均覆盖1—90步，没有记录到 NaN/Inf。
- 2. 梯度稳定。范数约0.029—0.043，远低于裁剪阈值1，没有梯度爆炸迹象。
- 3. 奖励和正确率有积极趋势。不是仅由格式改善解释。
- 4. 回答结束情况健康。没有记录到 backend abort 或空回答，整体截断比例约0.234%。
- 5. 训练负载均衡有效。第90步均衡后 token 总量相对极差约0.172%。
- 6. 已记录 actor 显存峰值近期平台化。但并不等于获得了所有进程、所有GPU的完整内存图景。

13.4 当前不能支持的结论

- 1. 不能说数学泛化能力已经提升：缺少独立验证结果。
- 2. 不能说模型更新后 KL 为0：当前诊断没有进行更新后复测。
- 3. 不能说师生完全一致：本次没有教师，蒸馏 KL 是占位。
- 4. 不能把 MFU≈2.2 解释为真实220%利用率或2.2%利用率。
- 5. 不能由 reward 最大值一直为1推断所有回答已正确。
- 6. 不能由 prompt clip_ratio=0推断原始数据没有长题目。
- 7. 不能由单一熵下降判断模式坍塌，也不能由单个正确率高点判断显著提升。

8. 不能由第1—90步记录推断第90步之后的当前运行状态。

13.5 后续观察优先级

目标	优先指标或动作	解释
判断数学训练表现	多窗口 <code>reward/math/acc</code>	不以单步高点代替趋势
区分格式与正确性收益	联合看 <code>score</code> 、 <code>format_score</code> 、 <code>acc</code>	避免只追综合奖励
判断泛化	固定独立验证集 / benchmark，保持生成与判题配置一致	当前训练曲线不能替代
观察探索与确定性	<code>entropy</code> 、 <code>student_mean_prob</code> 、同题六回答差异	低熵不自动等于更好
观察长度副作用	<code>response mean/max</code> 、 <code>clip_ratio</code> 、结束原因	长度增长需与收益联合评估
观察数值问题	<code>grad_norm</code> 、 <code>loss</code> 突变、 <code>NaN/Inf</code>	PG <code>loss</code> 不能照搬交叉熵的解释
优化运行效率	优先分析 rollout 长尾与每 token gen 时间	最近窗口生成等待约占69%
判断显存余量	每卡、各进程的实际设备内存与峰值	不只依赖 rank0 actor 统计
使用 MFU 做效率比较	先核实 L20X 峰值、模型 FLOPs 与计时口径	当前绝对数值不宜直接使用

上述是后续分析建议，不表示本报告已修改训练配置、重启任务或执行新的评测。

14. 代码依据与解释边界

代码定义以运行提交 `c35426965eabc4f1af17357d4e130af1e2149537` 为依据；下列路径相对仓库根目录，行号用于定位对应实现。

主题	主要代码位置
Math 训练配置	<code>hlg/mopd-idea/run_rl_math.sh:62</code>
奖励、结束原因与聚合	<code>plugin/open_mopd_rewards/common.py:56</code> 、 <code>:72</code> 、 <code>:98</code>
数学判题调用与外层超时	<code>plugin/open_mopd_rewards/math_reward.py:14</code>
Box 提取与数学检查	<code>plugin/open_mopd_rewards/vendor/math/__init__.py:40</code> 、 <code>plugin/open_mopd_rewards/vendor/math/math_utils.py:891</code>
GRPO 组内标准化	<code>verl/trainer/ppo/core_algos.py:304</code>
PPO loss、clipfrac、采样 KL	<code>verl/trainer/ppo/core_algos.py:1329</code>
Old logprob 与 entropy	<code>verl/trainer/main_ppo_sync.py:1377</code>

主题	主要代码位置
Actor 更新批次设置	verl/trainer/main_ppo_sync.py:1565
样本诊断范围与聚合	verl/trainer/main_ppo_sync.py:1664 、 :2069
蒸馏诊断和 student 概率	verl/trainer/main_ppo_sync.py:2186
批次指标组织与长度来源	verl/trainer/main_ppo_sync.py:2366
critic 与长度统计	verl/trainer/ppo/metric_utils.py:123
每 token 耗时与吞吐	verl/trainer/ppo/metric_utils.py:310 、 :353
Token 负载均衡	verl/trainer/main_ppo_sync.py:1351 、 verl/utils/seqlen_balancing.py:257
梯度范数和优化器更新	verl/workers/engine/fsdp/transformer_impl.py:659
Engine 指标、内存与 MFU	verl/workers/engine_workers.py:183 、 :210
FLOPs 设备表与 Qwen 估算器	verl/utils/flops_counter.py:22 、 :89
Metadata 返回结果合并	verl/utils/tensordict_utils.py:221
Single-turn 轮次标记	verl/experimental/agent_loop/single_turn_agent_loop.py:79

这是一次只读实验分析的文档化整理，没有修改模型、训练参数、远端进程或任务状态。报告没有单独保存全部 8,640个原始标量点，数值表是当时读取结果的汇总；后续复查应从同一 TensorBoard 或原始 event 文件按相同步骤范围获取。

最终结论：本次90步快照支持“训练正常且训练指标有所改善”，不支持直接宣称“数学泛化能力已经提升”。

附录：96 个指标完整清单

本报告按功能重新组织章节，原始 TensorBoard 的13个顶层分组及数量如下。

原始分组	数量
actor	11
critic	12
global_seqlen	6
perf	4
prompt_length	4
response	1
response_length	4
response_length_non_aborted	4
reward	26
timing_per_token_ms	3
timing_s	6
train_samples	10
training	5
合计	96

actor/entropy	critic/rewards/min
actor/grad_norm	critic/score/max
actor/loss	critic/score/mean
actor/lr	critic/score/min
actor/perf/cpu_memory_used_gb	global_seqlen/balanced_max
actor/perf/max_memory_allocated_gb	global_seqlen/balanced_min
actor/perf/max_memory_reserved_gb	global_seqlen/max
actor/pg_clipfrac	global_seqlen/mean
actor/pg_clipfrac_lower	global_seqlen/min
actor/pg_loss	global_seqlen/minmax_diff
actor/ppo_kl	perf/mfu/actor
critic/advantages/max	perf/throughput
critic/advantages/mean	perf/time_per_step
critic/advantages/min	perf/total_num_tokens
critic/returns/max	prompt_length/clip_ratio
critic/returns/mean	prompt_length/max
critic/returns/min	prompt_length/mean
critic/rewards/max	prompt_length/min
critic/rewards/mean	response/aborted_ratio

response_length/clip_ratio	reward/math/samples
response_length/max	reward/math/score
response_length/mean	reward/math/stopped_normally
response_length/min	reward/math/termination_known
response_length_non_aborted/clip_ratio	reward/math/test_pass_ratio
response_length_non_aborted/max	timing_per_token_ms/adv
response_length_non_aborted/mean	timing_per_token_ms/gen
response_length_non_aborted/min	timing_per_token_ms/update_actor
reward/all/acc	timing_s/adv
reward/all/answer_extracted	timing_s/gen
reward/all/checker_timeout	timing_s/old_log_prob
reward/all/constraint_ratio	timing_s/step
reward/all/format_score	timing_s/update_actor
reward/all/generation_aborted	timing_s/update_weights
reward/all/is_truncated	train_samples/advantage_mean
reward/all/known_termination_count	train_samples/advantage_std
reward/all/samples	train_samples/distill_kl_fwd_max
reward/all/score	train_samples/distill_kl_fwd_mean
reward/all/stopped_normally	train_samples/reward_max
reward/all/termination_known	train_samples/reward_mean
reward/all/test_pass_ratio	train_samples/reward_min
reward/math/acc	train_samples/reward_std
reward/math/answer_extracted	train_samples/student_entropy_mean
reward/math/checker_timeout	train_samples/student_mean_prob
reward/math/constraint_ratio	training/epoch
reward/math/format_score	training/global_step
reward/math/generation_aborted	training/num_turns/max
reward/math/is_truncated	training/num_turns/mean
reward/math/known_termination_count	training/num_turns/min